Politecnico di Milano

Ingegneria del Software – a.a. 2008/09

Appello del 17 Luglio 2009

Cognome Nome Matricola Sezione (segnarne una) Baresi ☐, Ghezzi ☐, San Pietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 6)

parte a

Si consideri il seguente metodo statico:

public static String maxSottostringa (String[] S)

Il metodo riceve come parametro un array di stringhe, che contiene almeno una stringa, e in cui tutte le stringhe non sono vuote.

Il metodo restituisce la sottostringa massimale comune, ossia la più grande sottostringa consecutiva comune a tutti gli elementi dell'array. Qualora non vi sia nessuna stringa comune (nemmeno di un singolo carattere) il metodo restituisce la stringa vuota "". Se vi sono più sottostringhe massimali della stessa lunghezza, il metodo ne restituisce una qualunque.

Ad esempio, se S = [acca accbb ccaba], il metodo restituisce la stringa cc.

Specificare **maxSottostringa(...)** in JML. Non è necessario che la specifica sia eseguibile. Per comodità si riportano alcuni metodi pubblici della classe String, con i relativi contratti estratti dalla libreria java:

int length(): Restituisce la lunghezza della stringa this.

public String substring(int beginIndex, int endIndex) Restituisce una nuova stringa che è una sottostringa della stringa, e che inizia al beginIndex specificato e si estende fino al carattereall'indice endIndex - 1. NB: Gli indici della stringa vanno da 0 a this.length()-1, come per gli array.
Throws: IndexOutOfBoundsException – se beginIndex<0 oppure endIndex>length() oppure beginIndex>endIndex.

Una semplice soluzione non eseguibile è:_

```
//@ requires S != null && S.length>0 &&
        (\forall int i; 0<= i && i< S.length; S[i]!=null && S[i].length>0)
//@ensures \result != null &&
    // \result è una sottostringa di ogni elemento di S:
    (\forall int h; 0<= i && i< S.length();
        (\exists int i; 0<= i && i<S[h].length();
            (\exists int j; i< j && j<=S[h].length; \result.equals(S[h].substring(i,j)))
    )    ) &&
    (\forall String y; y != null;
//ogni altra sottostringa di tutti gli elementi ha lungh.non superiore a quella di \result:
        (\forall int h; 0<= i && i< S.length;
            (\forall int i; 0<= i && i<S[h].length;
                (\forall int j; i< j && j<=S[h].length; y.equals(S[h].substring(i,j))
```

))) => y.length <= x.length)

parte b

Il metodo definito nella parte (a) definisce una funzione parziale o totale? Perché? Se la funzione non è totale, come si potrebbe modificare la specifica rendendo la funzione totale?

```
//@ requires true;

//@ensures  S.length>0 && (\forall int i; 0<= i && i< S.length; S != null &&
S[i].length>0)

    && \result != null &&

    // \result è una sottostringa di ogni elemento di S:

    (\forall int h; 0<= i && i< S.length();

        (\exists int i; 0<= i && i<S[h].length();

            (\exists int j; i< j && j<=S[h].length; \result.equals(S[h].substring(i,j)))

    )    ) &&

    (\forall String y; y != null;

//ogni altra sottostringa di tutti gli elementi ha lungh.non superiore a quella di \result:

        (\forall int h; 0<= i && i< S.length;

            (\forall int i; 0<= i && i<S[h].length;

                (\forall int j; i< j && j<=S[h].length; y.equals(S[h].substring(i,j))

                )          )          ) => y.length <= x.length )

// signals (NullPointerException e) S == null ||

                        (S.length>0 && (\exists int i; 0<= i && i< S.length; S[i] != null);

// signals (InvalidParameterException e) S != null && (S.length == 0 ||

                        (\exists int i; 0 <= i <= S.length; S[i] != null && S[i].length == 0));
```

Esercizio 2 (punti 14)

La classe **Corso** descrive un corso universitario. Ha tre metodi osservatori puri:

- **public int getCodice()**, che restituisce il codice del corso
- **public Docente getDocente()**, che restituisce il riferimento al docente
- **public boolean precedenza(Corso c)**, che restituisce un valore vero se il corso passato come parametro costituisce una precedenza obbligatoria per il corso in questione (e cioè il suo esame deve essere superato prima).

Il costruttore di **Corso** riceve come parametri un codice intero, un riferimento a **Persona** e un **ArrayList<Corso>** che contiene le precedenze obbligatorie.

La classe **Corso** è immutabile, ed in particolare non è possibile modificare l'insieme delle precedenze una volta che l'istanza del corso è stata creata.

Si consideri il seguente vincolo:

> *Un corso C non può essere precedenza di se stesso né può avere come precedenza un corso K che ha come precedenza il corso C.*

(a) Come si potrebbe esprimere questo vincolo come invariante pubblico di classe?

//@ public invariant (\forall Corso k; k != null;

this.precedenza(k) => ! k.precedenza(this));

(b) L'invariante è soddisfatto dalla specifica della classe? Se si, perché? Se no, che cosa bisogna fare perché sia soddisfatto?

Normalmente le specifiche dei metodi della classe devono supportare l'invariante di classe, e quindi bisogna gestire accuratamente i casi che potrebbero portare alla sua violazione. Per esempio, considerando un insieme di numeri senza duplicati, la specifica del metodo add(int num) deve escludere la possibilità di inserire un valore duplicato nell'insieme.

Tuttavia, in questo caso l'invariante di classe viene garantito per costruzione, in quanto l'elenco delle dipendenze viene richiesto nel costruttore del Corso. Per questo

- ***un corso non potrà mai avere come precedenza se stesso (dovrei passare nel costruttore un'istanza dell'oggetto che sto creando)***
- ***dato un corso C che ha come precedenza il corso K, K non può avere come precedenza il corso C. Nella costruzione di C, infatti, devo passare come parametro il riferimento al corso K, affinchè C abbia come precedenza K. Tut-***

tavia, affinché K abbia come dipendenza C, al momento della costruzione di K dovrei passare un riferimento al corso C, che non è ancora stato costruito.

Per questi motivi, non è strutturalmente possibile violare l'invariante pubblico.

La classe **Curriculum** descrive l'insieme degli esami superati da uno studente. Fornisce il seguente metodo osservatore puro:

- **public ArrayList<Corso> getCV()**, che restituisce gli esami superati

e i seguenti ulteriori metodi:

- **public Curriculum()**, il costruttore che inizializza il curriculum con zero esami sostenuti

- **public void addEsame(Corso c)**, che viene chiamato quando l'esame è stato superato, e quindi va aggiunto al curriculum. Il metodo deve lanciare l'eccezione **EsameIllegaleException** se il curriculum contiene già un esame superato con lo stesso codice dell'esame che si vuole inserire nel curriculum

- **public int getVoto(Corso c)**, che restituisce il voto ottenuto dallo studente nel corso specificato.

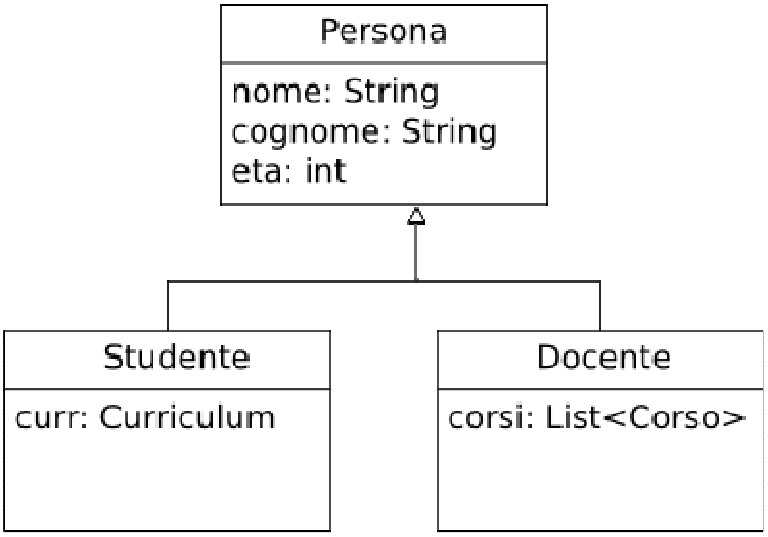
(c) Fornire la specifica JML del costruttore **Curriculum** e del metodo **addEsame()**.

```
//@ ensures getCV().size == 0;
public Curriculum();

//@requires c != null;
//@ensures (\forall int i; 0 <= i && i < \old(getCV().size());
                        \old(getCV().get(i).getCodice()) != c.getCodice()) &&
    (\forall int i; 0 <= i && i < getCV().size();
        getCV().get(i) == c ||
        (\exist int j; 0 <= j && j < old(getCV().size());
            \old(getCV().get(j)) == getCV().get(i) ) );
// signals (EsameIllegaleException e) (\exists int i; 0 <= i && i < \old(getCV().size());
                        \old(getCV().get(i).getCodice()) == c.getCodice());
```

Si ipotizzi di voler definire le classi **Studente** e **Docente**. Esse hanno attributi comuni (per esempio, nome, cognome, età), ma anche attributi aggiuntivi diversi: **Studente** ha un **Curriculum** e **Docente** ha un insieme di corsi.

(d) Come pensate di definire queste classi? Definite la struttura, senza scendere in dett***agli implementativi.***



Si supponga che una classe **CorsoDiLaure**a abbia due osservatori puri che consentono di estrarre gli studenti e i docenti del corso di laurea. I metodi sono rispettivamente:

- **public ArrayList <Docente> getDocenti**();
- **public ArrayList <Studente> getStudenti**();

È richiesto che i due osservatori rispettino il vincolo che i docenti degli esami superati dagli studenti del corso di laurea siano docenti dello stesso corso di laurea.

(e) Specificare i due osservatori in JML.

E' sufficiente he il vincolo sia imposto da uno solo dei due metodi:

```
//@ ensures \result != null
public ArrayList <Docente> getDocenti();

//@ ensures \result != null && (\forall int i ; 0 <= i && i < \result.size();
        (\forall int j; 0 <= j && j < \result.get(i).getCurr().getCV();
                (\exists int z; 0 <= z && z < getDocenti().size();
                        \result.get(i).getCurr().getCV().get(j).getDocente() ==
                        getDocenti().get(z) )));
public ArrayList <Studente> getStudenti();
```

Si supponga che la classe **Curriculum**, invece di esportare il metodo **getCV**, definisca un iteratore che consente di ottenere uno alla volta i corsi superati da uno studente.

(f) Definire in modo schematico la classe Curriculum in questa nuova versione.

```
class Curriculum implements Iterable<Corso> {
        private static class CurrIter implements Iterator<Corso> {
                public boolean hasNext() { … }
                public Corso next() { … }
                public void remove () { … }
        }

        public void addEsame(Corso c) { … }
        public Iterator<Corso> iterator() {
                return new CurrIter ();
        }
}
```

(g) Definire un metodo che calcola la media degli esami di un certo curriculum, usando l'iteratore.

```
//intestazione del metodo...
```

```
Curriculum cur // parametro del metodo
int tot = 0, num = 0;
for(Corso c : cur) {
        num ++;
        tot += cur.getVoto(c);
}
return ( (float) tot ) / num;
```

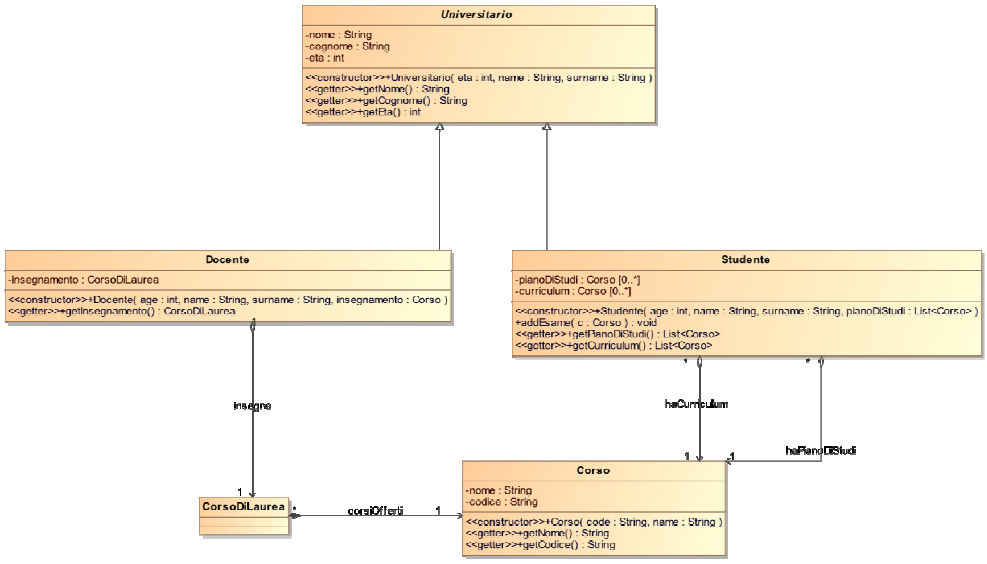
Esercizio 3 (punti 5)

Si formalizzi con un diagramma delle classi UML la situazione di cui all'esercizio precedente. Più precisamente, un'università è caratterizzata da corsi laurea, nei quali operano docenti e studenti. Ogni studente ha un curriculum di esami superati e ogni docente ha associati i corsi che insegna. Oltre al curriculum, ogni studente ha anche un piano di studi, e cioè un insieme di corsi ancora da superare.

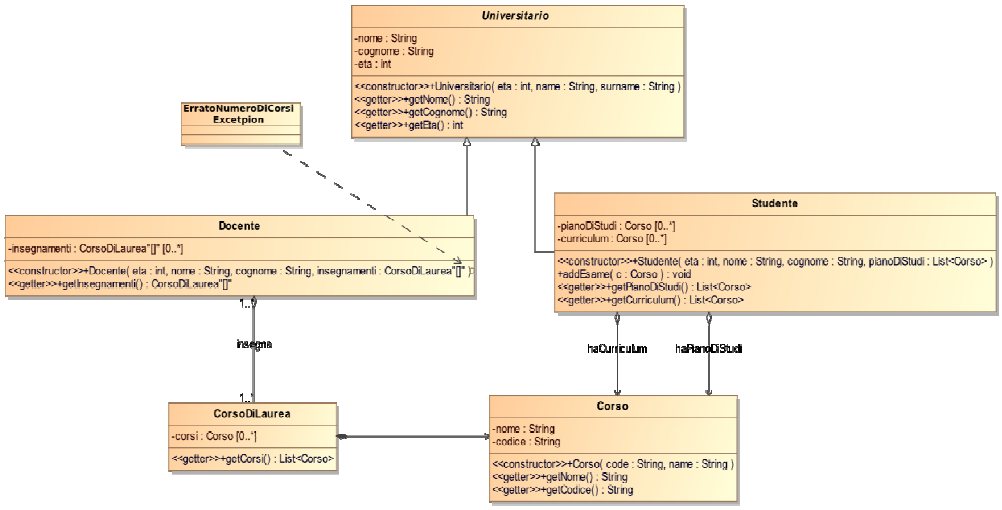
- Si dica esplicitamente che cosa cambia nel diagramma nelle seguenti ipotesi:

(a) ogni docente insegna in uno e uno solo corso di laurea, piuttosto che (b) un docente può insegnare in più corsi di laurea, ma almeno in uno e in non più di 3, piuttosto ***che (c) non esistono vincoli al numero di corsi laurea in cui un docente può insegnare.***

(a)

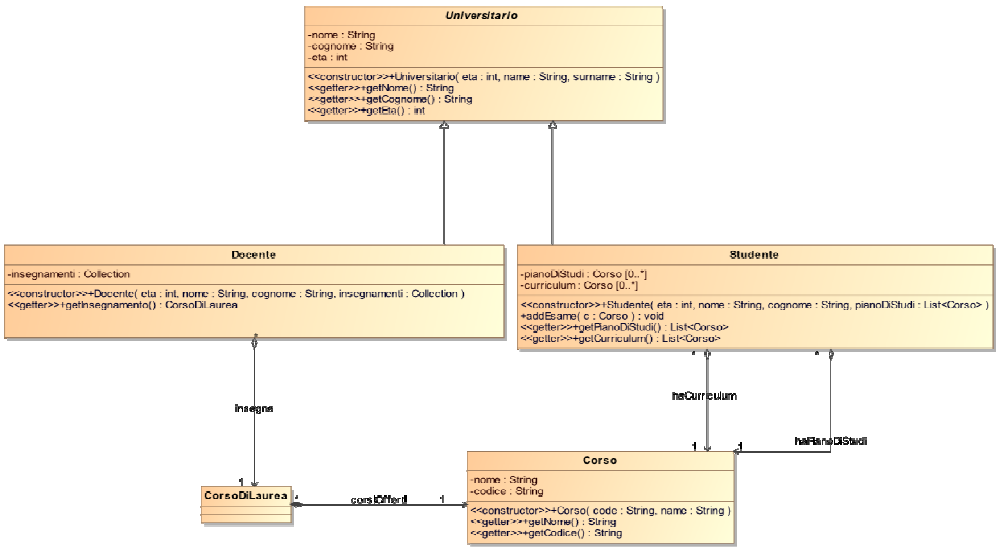


(b)



Si noti che UML non consente di specificare la molteplicità 1..3 per una relazione, per questo bisogna specificare comunque la molteplicità 1..* e poi inserire nel costruttore un controllo che l'array di CorsoDiLaurea passato come parametro non contenga mai più di 3 elementi e che non sia vuoto. Nel caso questo accada viene lanciata l'eccezionei ErratoNumeroDiCorsiException.

(c)



- Usando JML si esprima il vincolo che per ogni studente la somma degli esami del curriculum e degli esami del piano di studi deve essere 30.

Public class Studente extends Universitario {

//@public invariant getPianoDiStudi.size() + getCurriculum.size() == 30;

//corpo della classe

}